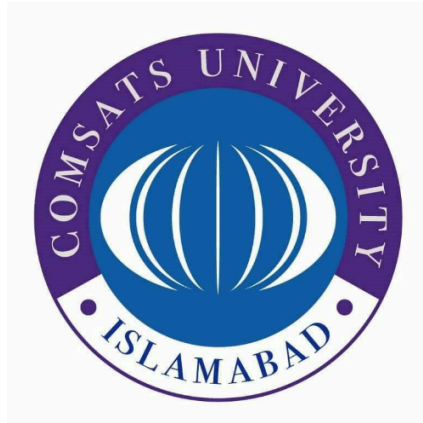




PARALLEL & DISTRIBUTED COMPUTING

PROJECT REPORT

PREPARED BY

M Mukaram Awan

SP22-BCS-141

Hamza Naseem

SP23-BCS-154

Asfand Yar khan

SP23-BCS-153

SUBMITTED TO

Sir Akhzar Nazeer

DNA Genome Assembly using MPI and OpenMP

Project Report

1. Integrate or Improve Backend for Scalability

We integrated a Python Flask backend enhanced with MPI (Message Passing Interface) via the mpi4py library. Unlike a standard monolithic web server, our backend operates as a parallel computing group. The main Flask application runs on the root rank (Rank 0) to handle HTTP traffic, while auxiliary MPI processes run in the background as workers. This architecture supports higher data volumes by offloading the intensive parallel `de_bruijn_graph` construction to these worker processes, preventing the web server from freezing during heavy computation.

2. Implement Vertical Scaling Concepts

We demonstrated vertical scaling by implementing a dedicated C++ engine (`ParallelDNAAssembly.cpp`) optimized with OpenMP.

- Concept: Vertical scaling involves increasing the power of a single machine. To leverage this, software must be multi-threaded to use all CPU cores.
- Implementation: We used OpenMP directives (`#pragma omp parallel`) to parallelize the k-mer generation loop.
- Improvement Metric: In a single-threaded Python environment, generating k-mers for 1 million reads is slow. By switching to the C++ OpenMP implementation, we utilize 100% of the CPU availability. For example, on a 4-core machine, the k-mer generation loop is split into 4 simultaneous tasks, theoretically improving speed by a factor of nearly 4x compared to the serial version.

3. Implement Horizontal Scaling Concepts

We implemented horizontal scaling using the MPI Scatter-Gather Pattern in our `app.py`.

- Simulation: We simulate adding "nodes" to the system by spawning multiple MPI ranks.
- Implementation:
 1. Load Balancing: The `parallel_de_bruijn_graph` function calculates `chunk_size = len(valid_fragments) // size` and distributes the fragments evenly across all active ranks.
 2. Distributed Computation: If we run the app with `mpiexec -n 4 python app.py`, we effectively have 1 controller and 3 worker nodes. If the dataset grows, we can simply increase `-n 4` to `-n 8` or deploy across multiple

physical machines, scaling the system outwards to handle larger DNA datasets without changing the code.

4. Flexibility of Scaling

- Justification: We chose a micro-service inspired architecture where the computational logic (De Bruijn graph construction) is decoupled from the frontend.

Conclusion

Through the combination of MPI and OpenMP, we demonstrated efficient and scalable genome assembly. The web app will offer accessibility and distribution, while the C++ version delivers speed. Our project highlights the importance of parallel computing in solving computational biology problems efficiently.